Multiscale Methods

Optimization of Discretized Continuous Functions

5 Jun 2026 | SIAM OP26 | Edinburgh, UK

Nicholas J. E. Richardson[1], Noah Marusenko[2], & Michael P. Friedlander[1,2]

Departments of [1]Mathematics and [2]Computer Science

INSTITUTE OF APPLIED MATHEMATICS
University of British Columbia

siam 2026 | Conference on Optimization

THE UNIVERSITY OF BRITISH COLUMBIA

The Problem

What we want

Solve

$$\min_f \{\mathcal{L}(f) \mid f \in \mathcal{C}\} \qquad (P)$$

with

- objective $\mathcal{L} : \mathcal{C} \to \mathbb{R}$
- continuous function $f : \mathcal{D} \to \mathbb{R}$
- domain $\mathcal{D} \subseteq \mathbb{R}$
- constraint $\mathcal{C} \subseteq \mathcal{F} := \{f : \mathcal{D} \to \mathbb{R} \mid f \text{ is continuous}\}$.

How can we solve P on a computer?

Some existing approaches

Solve over a (truncated) basis for f

- e.g. wavelets [Benedetto *et al.* 1993], polynomials [Trefethen 2019]

Parametrize f with an expressive family

- e.g. Gaussians [Vershynin 2018], neural networks [Gurney 2018]

Use variational calculus techniques [Gelfand *et al.* 2000]

Use duality [Chuna *et al.* 2025]

Some existing approaches

Solve over a (truncated) basis for f

- e.g. wavelets [Benedetto *et al.* 1993], polynomials [Trefethen 2019]

Parametrize f with an expressive family

- e.g. Gaussians [Vershynin 2018], neural networks [Gurney 2018]

Use variational calculus techniques [Gelfand *et al.* 2000]

Use duality [Chuna *et al.* 2025]

Cons: too much regularity needed!

- e.g. f is bandlimited, derivatives decay at some rate, many parameters needed, f and $\mathcal{L}$ are C^2, $\mathcal{L}$ and $\mathcal{C}$ have special structure

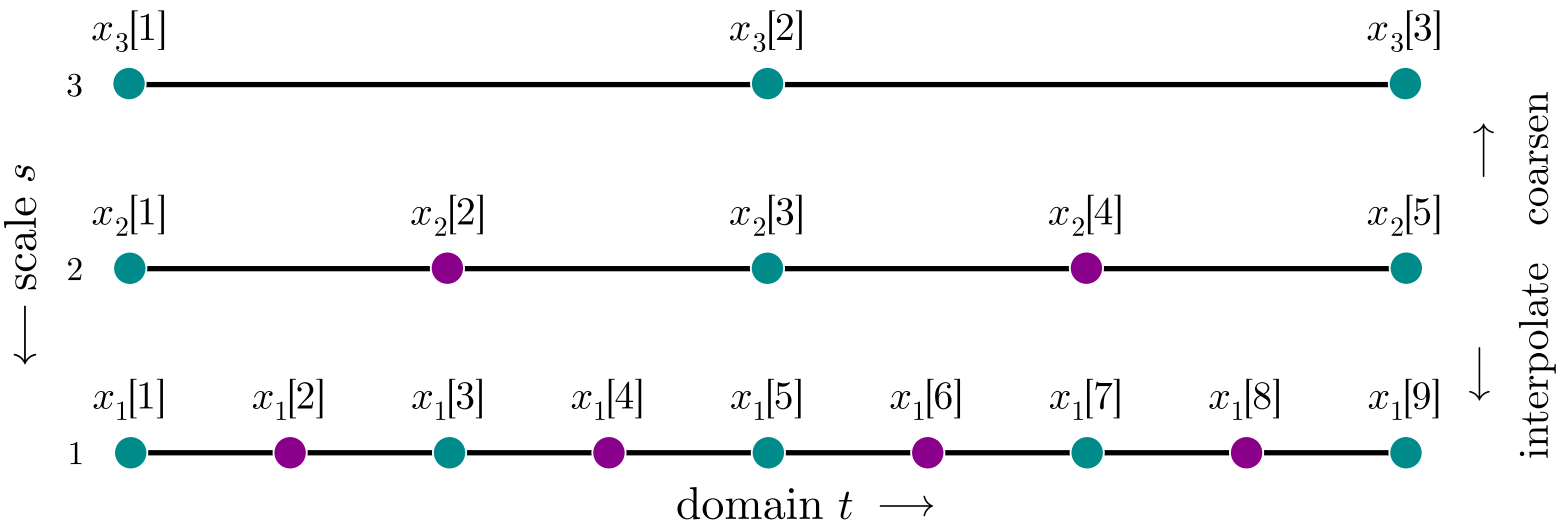
Why not just discretize the problem?

Continuous	Discrete
$\min_f \{\mathcal{L}(f) \mid f \in \mathcal{C}\} \quad (P)$	$\min_x \{\tilde{\mathcal{L}}(x) \mid x \in \tilde{\mathcal{C}}\} \quad (\tilde{P})$
f continuous	$x \in \mathbb{R}^I$
$f : \mathcal{D} \subseteq \mathbb{R} \to \mathbb{R}$	$x[i] = f(t[i])$
$\mathcal{L} : \{f : \mathcal{D} \to \mathbb{R}\} \to \mathbb{R}$	$\tilde{\mathcal{L}} : \mathbb{R}^I \to \mathbb{R}$
$\mathcal{C} \subseteq \{f : \mathcal{D} \to \mathbb{R}\}$	$\tilde{\mathcal{C}} \subseteq \mathbb{R}^I$

Works, but needs many points I or iterations for accurate solution!

Our approach

- Solve P on progressively finer discretizations of f (multiple *scales* s)
- *Related*: multigrid [Trottenberg *et al.* 2001], MGProx [Ang *et al.* 2024]



Example discretization with 3 scales. Sample $x_s[i] = f(t_s[i])$.

Start at big scales → go to smaller scales (s counts *down*)

Our approach

What's new?

- general framework: accelerates any nice* algorithm
- show iterate & computational analysis

What do we assume?

- Lipschitz minimizing solutions f^*
- *iterative updates, $x^{k+1} = U\left(x^k\right)$, that are q-linear for $0 < q < 1$:

$$\left\|x^{k+1} - x^*\right\|_2 \leq q\left\|x^k - x^*\right\|_2$$

You give me your favourite algorithm, here's how to speed it up.

A Numerical Example

The Multiscale Method: Example

Recover continuous probability density $f : [-1, 1] \to \mathbb{R}_+$ from polynomial measurements $y \in \mathbb{R}^M$ with regularized least-squares:

$$\min_f \left\{ \mathcal{L}(f) := \frac{1}{2}\|\mathcal{A}(f) - y\|_2^2 + \frac{1}{2}\lambda\|f'\|_2^2 \;\middle|\; \|f\|_1 = 1 \text{ and } f \geq 0 \right\}$$

with

- measurements $\mathcal{A}(f)[m] := \langle a_m, f \rangle = \int_{-1}^{1} a_m(t) f(t)\, dt$
- Legendre polynomials a_m of degree m
- regularizer $\|f'\|_2^2 = \int_{-1}^{1} f'(t)^2\, dt$ to promote smooth f.

The Multiscale Method: Discretized Example

Uniformly discretize $[-1, 1]$ with I points $t[i] = 2\frac{i-1}{I-1} - 1$ to get

$$\min_x \left\{ \tilde{\mathcal{L}}(x) := \frac{1}{2}\|Ax - y\|_2^2 + \frac{1}{2}\lambda x^\top G x \;\middle|\; \|x\|_1 = 1 \text{ and } x \geq 0 \right\}$$

where $x[i] = p(t[i])\Delta t$ and

$$A[m, i] = a_m(t[i]) \quad \text{and} \quad G = \frac{1}{\Delta t^3} \begin{bmatrix} 1 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 1 \end{bmatrix}.$$

The Multiscale Method: Discretized at Multiple Scale

Discretize at multiple scales $s = S, S-1, ..., 1$ to get

$$\min_{x_s} \left\{ \tilde{\mathcal{L}}_s(x_s) := \frac{1}{2}\|A_s x_s - y\|_2^2 + \frac{1}{2}\lambda x_s^\top G_s x_s \;\middle|\; \|x_s\|_1 = \frac{1}{c_s} \text{ and } x_s \geq 0 \right\}$$

where

- $c_s = 2^{s-1}$
- $I_s = \{1 + 0c_s, 1 + 1c_s, 1 + 2c_s, ..., I\}$ (every c_s points)
- $x_s = x[I_s], \quad A_s = c_s A[:, I_s] \quad \text{and} \quad G_s = \frac{1}{c_s} G[I_s, I_s].$

Scale discretization so that $\mathcal{L}(f) \approx \tilde{\mathcal{L}}(x) \approx \tilde{\mathcal{L}}_s(x_s)$ for all scales s.

Multiscale Solution Method

Run 1 step of projected gradient descent on x_s^k at scales $s = 10, 9, ..., 2$, then run projected gradient descent on x_1^k for many iterations.

Multiscale Solution Method

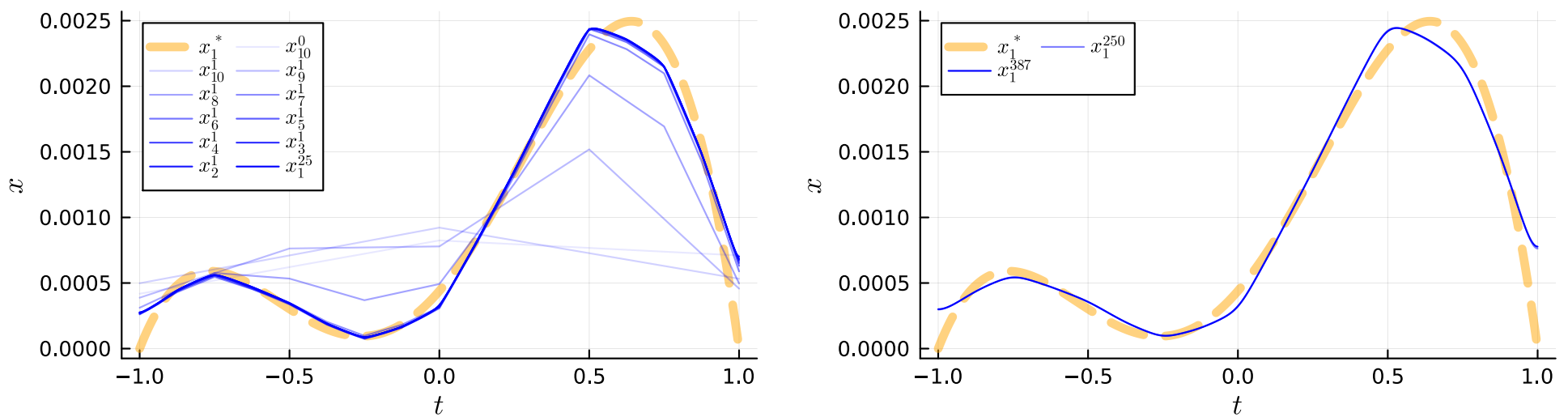
Run 1 step of projected gradient descent on x_s^k at scales $s = 10, 9, ..., 2$, then run projected gradient descent on x_1^k for many iterations.

The trick: Linearly interpolate x_s^1 to warm start the initialization x_{s-1}^0.

Multiscale Solution Method

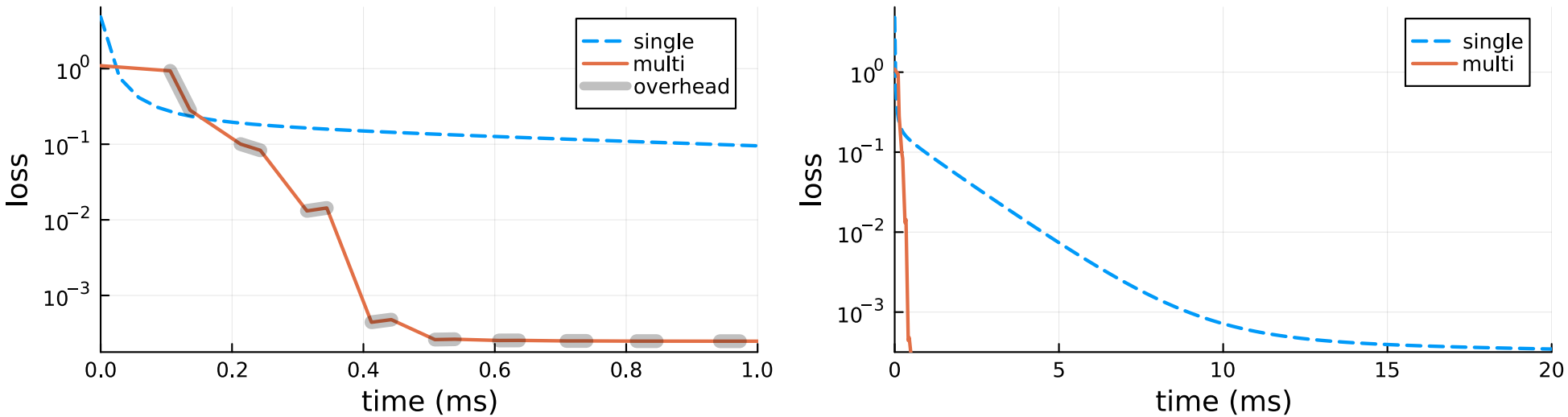
Run 1 step of projected gradient descent on x_s^k at scales $s = 10, 9, \ldots, 2$, then run projected gradient descent on x_1^k for many iterations.

The trick: Linearly interpolate x_s^1 to warm start the initialization x_{s-1}^0.



First few iterates of x_s^k (left), later iterates of x_s^k (right).

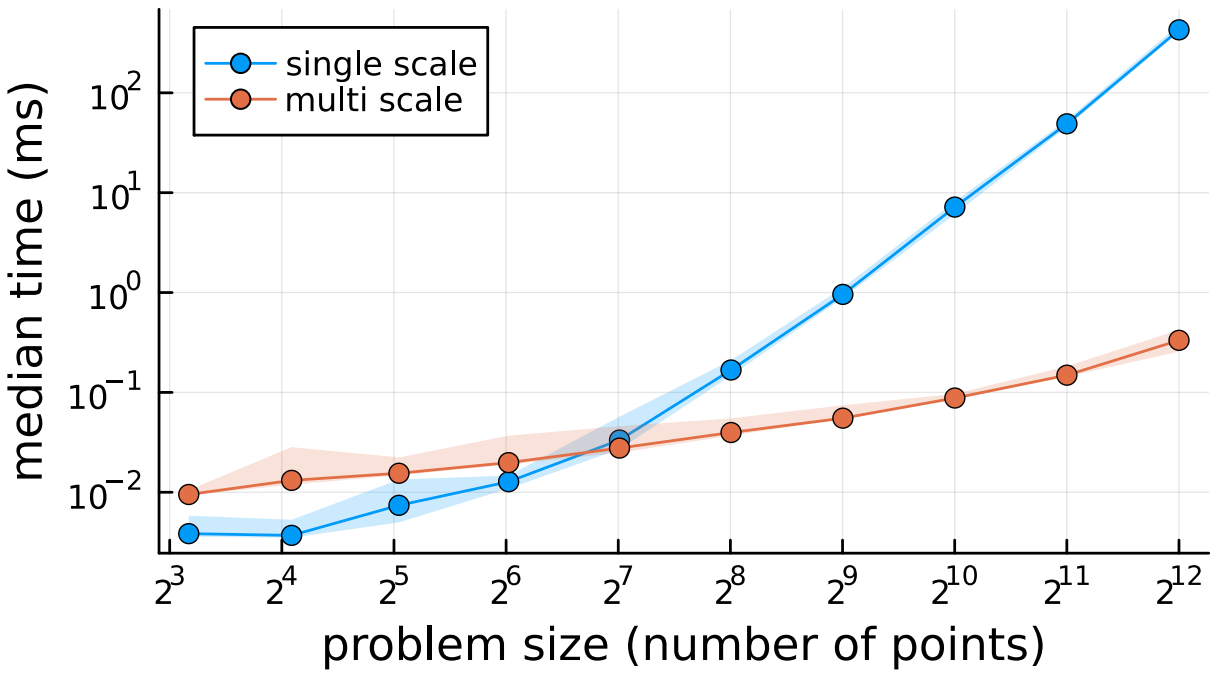
Faster you say?



Objective function $\tilde{\mathcal{L}}$ convergence over time. Grey regions highlight overhead: interpolation, allocations, and extra function calls.

How does it scale?

For large problems, turns seconds to milliseconds!



Median convergence times (within $5\,\%$ of optimal value) from 100 random initializations for problem size I.

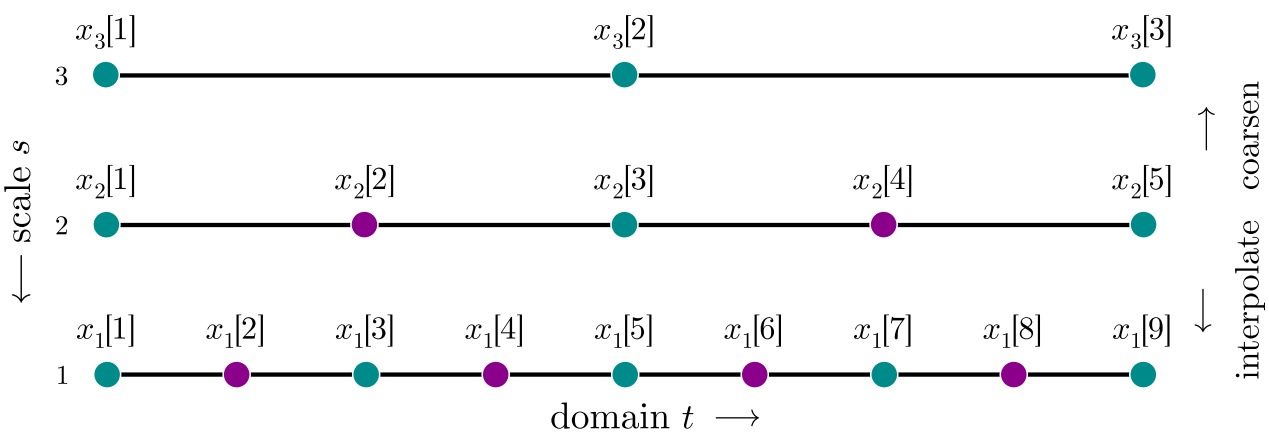
The Theory

Sufficient Conditions For...

...Iterate Convergence

- Solution functions $f^* \in \arg\min_{f \in \mathcal{C}} \mathcal{L}(f)$ are L_{f^*}-Lipschitz
- Run K_s iterations at scale s using a q-linear update $x_s^{k+1} = U\left(x_s^k\right)$,

$$\left\| x^{k+1} - x^* \right\|_2 \leq q \left\| x^k - x^* \right\|_2$$



Multiscale Iterate Convergence

Theorem 1: *The multiscale method returns an iterate x_1^k after k iterations satisfying*

$$\left\| x_1^k - x_1^* \right\|_2 \leq C^k \left\| x_S^0 - x_S^* \right\|_2 + D^k$$

where $C^k, D^k \longrightarrow 0$ at a rate depending on the q-linearity of the update $U(x)$, Lipschitz constant of the solution L_{f^*}, and # of scales S.

Multiscale Iterate Convergence

Theorem 1: *The multiscale method returns an iterate x_1^k after k iterations satisfying*

$$\left\| x_1^k - x_1^* \right\|_2 \leq C^k \left\| x_S^0 - x_S^* \right\|_2 + D^k$$

where $C^k, D^k \longrightarrow 0$ at a rate depending on the q-linearity of the update $U(x)$, Lipschitz constant of the solution L_{f^*}, and # of scales S.

Final error at the finest scale $s = 1$ is bounded

Multiscale Iterate Convergence

Theorem 1: *The multiscale method returns an iterate x_1^k after k iterations satisfying*

$$\left\|x_1^k - x_1^*\right\|_2 \leq C^k \left\|x_S^0 - x_S^*\right\|_2 + D^k$$

where $C^k, D^k \longrightarrow 0$ at a rate depending on the q-linearity of the update $U(x)$, Lipschitz constant of the solution L_{f^*}, and # of scales S.

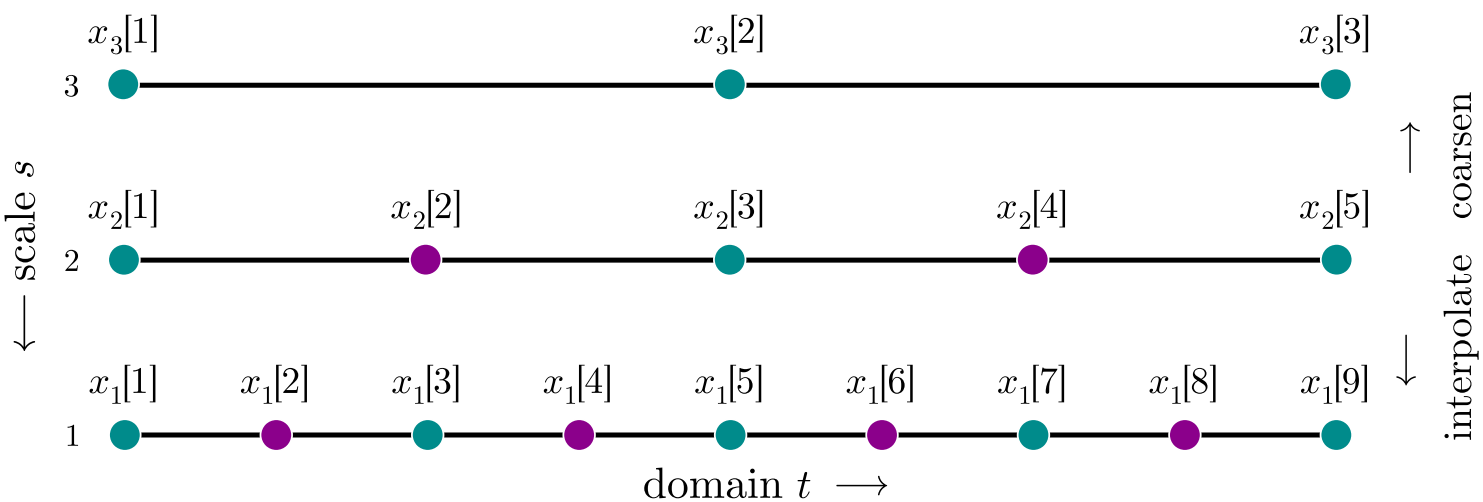
Final error at the finest scale $s = 1$ is bounded by the sum of the

- initial error
- linear interpolation error

Sufficient conditions for...

...Multiscale To Beat Single Scaled

- Initialize with i.i.d. standard normal entries $\mathcal{N}(0,1)$ (i.e. use randn)
- Cost of U scales at best linearly in the # of points: $C \geq \Omega(I)$
- Run one iteration $K_s = 1$ for coarse scales $s = S, S-1, ..., 2$



Conditions for Multiscale to be Better Than Single

Theorem 2: *If you perform multiscale with S scales where*

$$2^S \geq \frac{L_{f^*}|u-\ell|}{\text{poly}(q)},$$

then, multiscale simultaneously achieves lower total cost (i.e. faster) and a tighter final error bound (i.e. more accurate) than single scale.

For small q (easier problems!), need *more scales* for it to be worth it.

Conditions for Multiscale to be Better Than Single

Theorem 2: *If you perform multiscale with S scales where*

$$2^S \geq \frac{L_{f^*}|u-\ell|}{\text{poly}(q)},$$

then, multiscale simultaneously achieves lower total cost (i.e. faster) and a tighter final error bound (i.e. more accurate) than single scale.

For small q (easier problems!), need *more scales* for it to be worth it.

Note $q = 0 \implies$ can solve the fine scale problem in one iteration.

But does the discrete solution approximately solve P?

Yes!

Theorem 3: *Let f^* have uniform discretization $x^* \in \mathbb{R}^I$. Construct the piecewise linear function $\hat{f}$ from $x_1^{K_1} \in \mathbb{R}^I$. Then $\forall \epsilon > 0$,*

$$\left\| x_1^{K_1} - x^* \right\|_2^2 < C\epsilon \quad \text{and} \quad I > D\epsilon^{-1} + 1 \implies \left\| \hat{f} - f^* \right\|_2 < \epsilon$$

for constants $C, D > 0$ depending the Lipschitz constant of f^*.

But does the discrete solution approximately solve P?

Yes!

Theorem 3: *Let f^* have uniform discretization $x^* \in \mathbb{R}^I$. Construct the piecewise linear function $\hat{f}$ from $x_1^{K_1} \in \mathbb{R}^I$. Then $\forall \epsilon > 0$,*

$$\left\| x_1^{K_1} - x^* \right\|_2^2 < C\epsilon \quad \text{and} \quad I > D\epsilon^{-1} + 1 \implies \left\| \hat{f} - f^* \right\|_2 < \epsilon$$

for constants $C, D > 0$ depending the Lipschitz constant of f^*.

Solve the discrete problem accurately

But does the discrete solution approximately solve P?

Yes!

Theorem 3: *Let f^* have uniform discretization $x^* \in \mathbb{R}^I$. Construct the piecewise linear function $\hat{f}$ from $x_1^{K_1} \in \mathbb{R}^I$. Then $\forall \epsilon > 0$,*

$$\left\| x_1^{K_1} - x^* \right\|_2^2 < C\epsilon \quad \text{and} \quad I > D\epsilon^{-1} + 1 \implies \left\| \hat{f} - f^* \right\|_2 < \epsilon$$

for constants $C, D > 0$ depending the Lipschitz constant of f^*.

Solve the discrete problem accurately with enough points

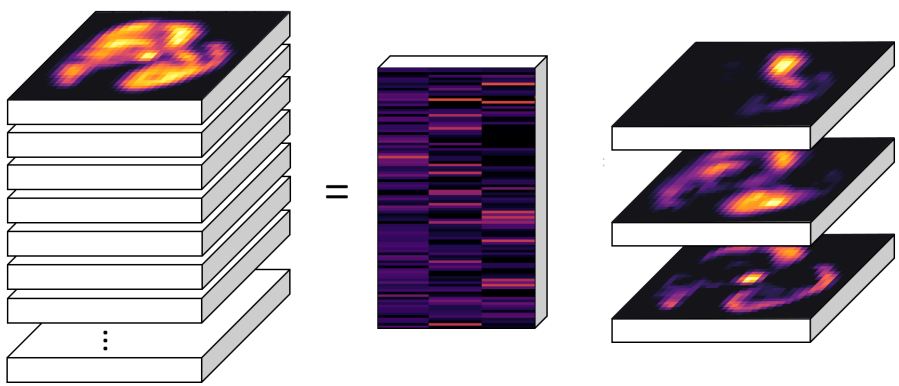
Example:
Tensor Factorization

Factorizing at Multiple Scales (see [Gillis *et al.* 2012])

Separate mixtures $Y[i,:,:,:]$ of 3D densities $B[r,:,:,:]$ by solving

$$\min_{A,B_s} \|Y_s - AB_s\|_F^2 \quad \text{s.t.} \quad A, B_s \geq 0, \|A[i,:]\|_1 = 1, \text{and } \|B_s[r,:,:,:]\|_1 = \frac{1}{\sqrt[3]{c_s}}$$

at scales $s = S, S-1, ..., 1$ using progressively finer resolution of slices.



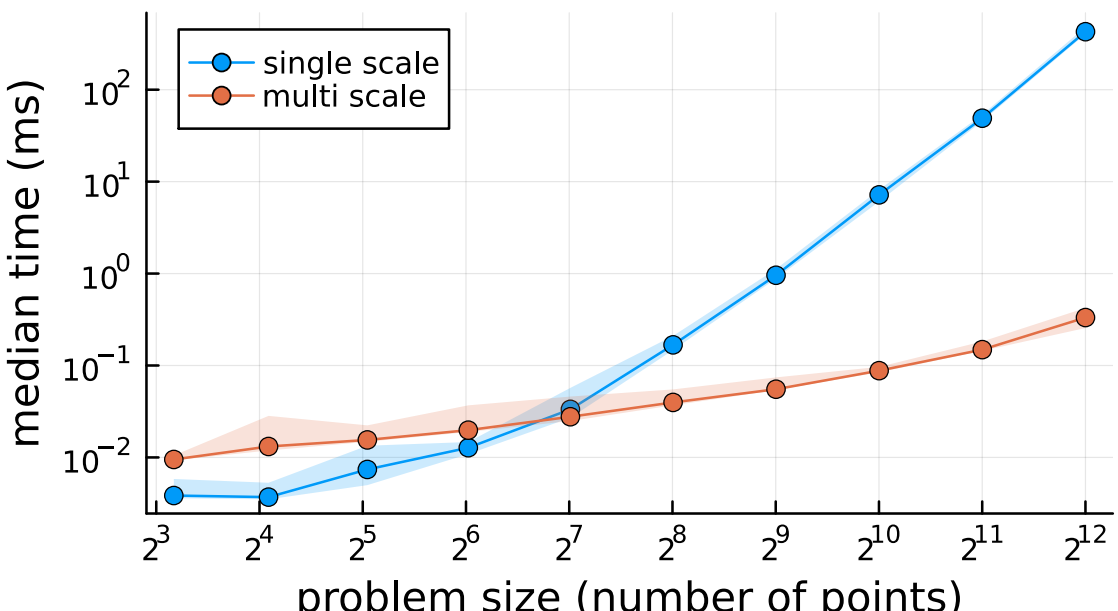
Example of a Tucker-1 factorization for a stack of 2D densities.

Handling Constraints At Multiple Scales

Type	Continuous $\mathcal{C}$	Discrete $\tilde{\mathcal{C}}_s$
nonnegative	$f : \mathcal{D} \to \mathbb{R}_+$	$x_s \geq 0$
restricted range	$f : \mathcal{D} \to \mathcal{R} \subseteq \mathbb{R}$	$x_s \in \mathcal{R}^{I_s}$
p-norm	$\|f\|_p = c$	$\|x_s\|_p = c \cdot (I_s/I_1)^{1/p}$
linear	$\langle g, f \rangle = c$ $\left(\int_{\mathcal{D}} g(t) f(t)\, dt\right)$	$\langle a, x_s \rangle = c \cdot (I_s/I_1)$ $a[i] = g(t[i])\sqrt{\Delta t}$ $x_s[i] = f(t[i])\sqrt{\Delta t}$

Multiscale Summary

- Can speed up algorithms when optimizing over continuous functions
- Uses approximate coarse solutions to warm start fine scale iterates
- Better than iterating at one scale when you want accurate solutions



Thank You!

Paper [Richardson *et al.* 2025]

https://arxiv.org/abs/2512.13993

Slides

https://njericha.github.io/research

References

[1] J. J. Benedetto and M. W. Frazier, *Wavelets: Mathematics and Applications*, 1st ed. Boca Raton: CRC Press, 1993.

[2] L. N. Trefethen, *Approximation theory and approximation practice*, Extended edition., no. 164. Philadelphia: Society for Industrial, Applied Mathematics, 2019.

[3] R. Vershynin, *High-Dimensional Probability*. University of California, Irvine: Cambridge University Press, 2018.

[4] K. Gurney, *An Introduction to Neural Networks*. London: CRC Press, 2018.

[5] I. M. Gelfand and S. V. Fomin, *Calculus of variations*. Mineola, N.Y: Dover Publications, 2000.

[6] T. Chuna, N. Barnfield, T. Dornheim, M. P. Friedlander, and T. Hoheisel, "Dual formulation of the maximum entropy method applied to analytic continuation of quantum Monte Carlo data," Jan. 2025. http://arxiv.org/abs/2501.01869

[7] U. Trottenberg, C. W. Oosterlee, and A. Schuller, *Multigrid Methods*. Academic Press, 2001.

[8] A. Ang, H. De Sterck, and S. Vavasis, "MGProx: A Nonsmooth Multigrid Proximal Gradient Method with Adaptive Restriction for Strongly Convex Optimization," *SIAM Journal on Optimization*, vol. 34, no. 3, Sept. 2024.

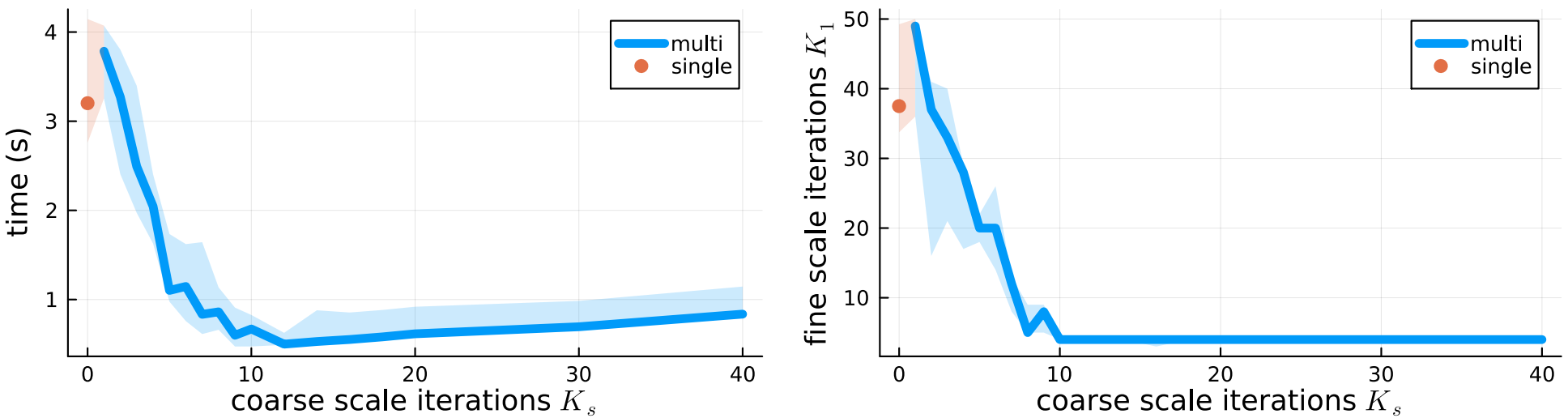
[9] N. Gillis and F. Glineur, "A multilevel approach for nonnegative matrix factorization," *Journal of Computational and Applied Mathematics*, vol. 236, no. 7, Jan. 2012.

[10] N. J. E. Richardson, N. Marusenko, and M. P. Friedlander, "Multiple Scale Methods For Optimization Of Discretized Continuous Functions," Dec. 2025. http://arxiv.org/abs/2512.13993

Additional Slides

Convergence vs Number of Coarse Scale Iterations

Allowing for multiple inner iterations $K_s \implies$ drop from $2.4\,\mathrm{s}$ to $0.3\,\mathrm{s}$!



(Left) Median time (20 trials) to convergence at the finest scale vs number of fixed iterations K_s at coarser scales $s = S, S-1, ..., 2$.
(Right) Median number of iterations K_1 at the finest scale $s = 1$.

The Multiscale Algorithm

Algorithm 1: Line 6 specifies Greedy or Lazy versions.

1: Randomly initialize x_s^0 at the coarsest scale $s = S$.
2: Perform K_S iterations on x_S^0 to approx. solve P_S and obtain $x_S^{K_S}$.
3: **for** scales $s = S-1, S-2, ..., 1$ **do**
4: Interpolate $x_{s+1}^{K_{s+1}}$ to obtain $\underline{x}_{s+1}^{K_{s+1}}$ according to Definition 2.
5: Initialize next scale using previous solution: $x_s^0 = \underline{x}_{s+1}^{K_{s+1}}$.
6: Perform K_s iterations on x_s^0 (Greedy) or $x_{(s)}^0$ (Lazy) to approximately solve P_s at scale s and obtain $x_s^{K_s}$.
7: **end**
8: **return** Approximate solution $x_1^{K_1}$ for P_s at $s = 1$

Coarsening and Interpolating

Definition 1 (Coarsening): *Dyadic coarsening maps a vector* $x \in \mathbb{R}^I$ *to* $\overline{x} \in \mathbb{R}^{\lfloor (I+1)/2 \rfloor}$ *by selecting odd-indexed entries:*

$$\overline{x}[i] = x[2i-1], \qquad i = 1, ..., \lfloor (I+1)/2 \rfloor.$$

Definition 2 (Interpolating): *Midpoint linear interpolation inserts a point between entries in* $x \in \mathbb{R}^I$ *to produce* $\underline{x} \in \mathbb{R}^{2I-1}$*:*

$$\underline{x}[i] = \begin{cases} x\left[\frac{i+1}{2}\right] & \text{if } i \text{ is odd,} \\ \frac{1}{2}\left(x\left[\frac{i}{2}\right] + x\left[\frac{i}{2}+1\right]\right) & \text{if } i \text{ is even.} \end{cases}$$

Continuous vs Discretize Problem

Continuous	Discrete
$\min_f \{\mathcal{L}(f) \mid f \in \mathcal{C}\} \quad (P)$	$\min_{x_s} \{\tilde{\mathcal{L}}_s(x_s) \mid x_s \in \tilde{\mathcal{C}}_s\} \ (P_s)$
f continuous	$x_s \in \mathbb{R}^{I_s}$
$f : \mathcal{D} \subseteq \mathbb{R} \to \mathbb{R}$	$x_s[i] = f(t_s[i])$
$\mathcal{L} : \{f : \mathcal{D} \to \mathbb{R}\} \to \mathbb{R}$	$\tilde{\mathcal{L}}_s : \mathbb{R}^{I_s} \to \mathbb{R}$
$\mathcal{C} \subseteq \{f : \mathcal{D} \to \mathbb{R}\}$	$\tilde{\mathcal{C}}_s \subseteq \mathbb{R}^{I_s}$

Updates with q-Linear Convergence

Definition 3 (q-Linear Iterate Convergence): *An iterative algorithm with update rule $x^{k+1} = U\left(x^k\right)$ has global q-linear iterate convergence with rate $q \in [0, 1)$ when*

$$\left\|x^k - x^*\right\|_2 \leq (q)^k \left\|x^0 - x^*\right\|_2$$

for any initialization $x^0 \in \tilde{\mathcal{C}}$ and minimizer x^* of $\tilde{\mathcal{L}}(x)$ over $\tilde{\mathcal{C}}$ that may depend on x^0.

Example: projected gradient descent with stepsize $\frac{2}{\mathcal{S}+\mu}$ for $\mathcal{S}$-smooth and μ-strongly convex objective $\tilde{\mathcal{L}}$, and convex $\tilde{\mathcal{C}}$.

Multiscale Iterate Convergence (details)

Theorem 4: *The multiscale method returns an iterate* $x_1^{K_1}$ *satisfying*

$$\left\|x_1^{K_1} - x_1^*\right\|_2 \leq \sqrt{2^{S-1}}(q)^{r_S}\left\|x_S^0 - x_S^*\right\|_2 + L_{f^*}\frac{|u-\ell|}{2\sqrt{2^{S+1}}}\sum_{s=1}^{S-1} 2^s (q)^{r_s},$$

where $r_s = \sum_{t=1}^{s} K_t$ *is the cumulative iteration count through scale* s.

Final error is bounded by the sum of the

- inner algorithm rate q times the initial error
- cumulative linear interpolation error from each scale

Multiscale is be Better Than Single (details)

Theorem 5: *Assume the finest scale problem has $I = 2^S + 1$ points, where the number of scales S satisfies*

$$S \geq \max\left\{4, \log_2\left(\frac{L_{f^*}|u-\ell|}{\sqrt{2}(q)^2(1-2q)\left(1-\sqrt{2}q\right)}\right)\right\}.$$

Then, multiscale simultaneously achieves lower total cost (i.e. faster) and a tighter expected final error bound (accurate) than single scale.

For small q (easier problems!), need $I \gtrsim \frac{1}{q^2}$ for multiscale to be worth it.
Note $q = 0 \implies$ can solve the fine scale problem in one iteration.

Approximately solving the continuous problem (details)

Theorem 6: *Let f^* have uniform discretization $x^* \in \mathbb{R}^I$. Construct the piecewise linear function $\hat{f}$ from $x_1^{K_1} \in \mathbb{R}^I$. Then $\forall \epsilon > 0$,*

$$\left\| x_1^{K_1} - x^* \right\|_2^2 < \sqrt{\frac{3}{40}}(u-\ell)L_{f^*}\epsilon \quad \textit{and} \quad I > \sqrt{\frac{8}{15}}(u-\ell)L_{f^*}\epsilon^{-1} + 1$$

$$\implies \left\| \frac{\hat{f} - f^*}{u-\ell} \right\|_2 < \epsilon.$$

Legendre Polynomials

In the motivating example, we use normalized polynomials,

$$a_m(t) = \sqrt{\frac{2m+1}{2}} \sum_{k=0}^{m} \binom{m}{k} \binom{m+k}{k} \left(\frac{t-1}{2}\right)^k,$$

where the binomial coefficient

$$\binom{m}{k} = \frac{m!}{k!(m-k)!}.$$

Legendre Polynomials